

CONTENTS IN DETAIL

ACKNOWLEDGMENTS

xv

INTRODUCTION

xvii

What Is a Bug?	xviii
Why We Should Fix Bugs	xviii
Debugging Is Inevitable	xix
Debugging Is Your Job	xx
Who This Book Is For	xxi
Prerequisites	xxi
How to Use This Book	xxi
Manage Expectations	xxii

THE WORKFLOW

1

Step 1: Reproduce	2
Step 2: Probe	3
Step 3: Examine	3
Step 4: Fix	4
Handle Bugs as a Team	5
Common Root Causes	6
Bad Data	6
Bad Pointers	7
Bad Logic	7
Edge Cases	7
Floating-Point Inaccuracies	7
Stale Code or Assets	8
Wrong Timing or Order	8
Unclear or Bad Requirements	8
User Experience Issues	9
Summary	9

PART I: REPRODUCE

11

What Makes a Great Bug Report?	11
1. Reproduce the Bug Yourself	15
Re-create the User Experience	15
Emulate the Reporter	16
Introduce Variations	17
Beware of Networking Complexity	18
2. Shorten the Feedback Loop	19
Reduce Build Times	19
Reduce Deployment Times	23
Reduce Startup Times	26
Jump to the Correct Location	29

3. Automate the Reproduction	29
Whether to Automate	29
How to Automate	30
When You Can't Automate.	34
Summary	35
 PART II: PROBE	 37
4. Try Shortcuts	38
Make Educated Guesses	39
Focus on Suspicious Systems	43
Ignore Irrelevant Systems	45
Know When to Change Your Strategy	47
5. Search the Internet	47
Don't Scroll Too Far	48
Try Different Search Engines	49
Use Advanced Search Engine Features	50
Search for Source Code.	51
Don't Forget Your Intranet.	52
6. Check Version Histories.	52
Find a Working Version	53
Decide What to Test	55
Identify the Offending Change	56
Look into the Future	56
Look into Branches and Forks	58
7. Attach the Debugger by Default	59
Inspect Your Program's Behavior	59
Use Exception Handling	60
Use Breakpoints to Inspect Your Code's Execution	61
Set Up Debuggers on Difficult Platforms	61
Handle Cases Where the Debugger Interferes	62
8. Instrument the Program to Report Problems	62
Use Assertions	63
Handle Errors	65
Signal Errors to the Caller	67
Use Obvious Test Patterns.	67
Don't Break the Reproduction with Modifications.	68
Make the Instrumentation Optional	69
9. Remove Randomness	69
Create Deterministic Programs	70
Disable Potential Sources of Randomness	72
Put Determinism to Work	78
Consider Rare Hardware Issues	79
10. Ask Colleagues for Help	79
Ask Your Team	80
Ask for External Support	82
11. Search in All Files	86
Choose and Configure Your Search Tool	86
Choose Useful Search Terms.	87
Search Beyond the Source Code.	88
Peek into Nontext Files.	89

Look for Strings Composed in Code	90
Use Searchable Patterns in Code	90
12. Collect Crash Dumps	91
Trigger a Crash Dump	91
Get Information from a Crash Dump	92
Implement Crash Dumps	92
Turn Crash Dumps into Actionable Data	95
Use Third-Party Tools to Simplify Crash Reporting	96
Summary	97
 PART III: EXAMINE	 99
13. Step Through the Code	100
Choose the Right Build Configuration	100
Start the Debugging Process	104
Use Advanced Debugging Features	111
14. Read the Code	113
Act Like a Virtual Machine	113
Strive for Readable Code	115
Record What You Find	117
Know Your Programming Languages	117
15. Follow the Data	122
Find the Original Data	123
Spot the Symptoms of Bad Data	123
Find Where the Data Becomes Bad	126
Compare Good and Bad Data	127
Follow Good Data	127
16. Question Everything	127
Find the Cause That Must Exist	128
Don't Trust Blindly	128
Investigate Assumptions and Dependencies	129
Verify Your Assumptions	133
Learn About the Context	133
17. Experiment	134
Conduct Exploratory Code Modifications	134
Refine and Test Hypotheses	135
Restore Your Code to Its Original State	140
18. Keep a Debugging Log	140
Track Your Progress	141
Avoid Distractions	143
Use Documentation Tools	144
Keep an Archive	145
Use Chats as a Log	145
19. Build Your Own Tools	146
Implementation	147
Recommended Features for Debugging Tools	149
Promotion and Documentation	152
Release Considerations	152
20. Build a Minimal Repro	153
Choose a Minimization Method	153
Minimize the Input	156

Share the Example	157
Prevent Regressions	158
21. Debug with printf	158
Avoid printf Pitfalls	159
Know When to Use printf.	159
Implement printf.	160
Leverage Advanced Logging	162
Clean Up	163
22. Use Off-the-Shelf Tools	164
Consider Availability	165
Know When to Reach for a Specialized Tool	165
23. Talk It Through	173
Define Your Goal	174
Choose Who You Talk To	174
Plan the Logistics.	176
Decide What to Cover.	177
Summary	178
 PART IV: FIX	 179
What Makes a Great Bug Fix	180
Make It Reversible	181
Address the Root Cause.	181
Verify.	182
24. Just Fix It	182
Remove the Root Cause	182
Change the Original Sources	183
Test for Side Effects	185
25. Share What You've Learned	186
Document the Fix	186
Find What Produced the Root Cause	187
Prevent the Bug from Happening Again	189
26. Search for Related Problems	191
Understand Patterns and Dependencies	191
Make Bulk Changes with Caution	195
27. Fix Early.	196
Assign Reasonable Priorities.	197
Leverage Context	197
Prepare for Attrition.	198
Avoid Dependent Behavior.	199
Take Ownership	200
28. Deploy a Quick Fix.	201
Undo the Offending Change	201
Implement a Workaround.	203
Communicate Workarounds	206
Don't Stop at the Workaround	206
29. Start Over	206
Rewrite the Offending Functionality	207
Test the New Implementation	210
Compare the Results	211
Stub the Implementation.	212

30. Find a Creative Solution212
 Declare It a Feature 213
 Remove the Feature 214
 Remember to Communicate 217
 Just Let It Be 219
Summary 219

CONCLUSION 221

INDEX 227